CloudWave 포트폴리오 — OliveYoung 세일 이벤트 시스템

🔵 프로젝트 요약

한 줄 정의

> OliveYoung 세일 러시 150,000 VU를 Kafka 3-Broker Non-blocking Retry + Karpenter Spot 자동 스케일링으로 OOMKilled 0건·서비스 중단 0건으로 처리하고, Version A/C 비교 실험으로 브로커 장애 시 P95 3,137ms → 436ms를 실측 데이터로 증명한 인프라 엔지니어.

역할

- Terraform 16개 모듈 IaC 설계 및 전체 AWS 인프라 구축
- Karpenter 전환 (16개 연쇄 에러 해결)
- KEDA 복합 트리거 설계 및 스케일링 정책 튜닝
- GitLab CI/CD 8단계 파이프라인 구성
- K8s 매니페스트 관리 (Deployment, KEDA, Karpenter, Istio)

시스템 아키텍처

```
사용자
  │ HTTPS
  ▼
CloudFront ─── S3 (프론트엔드 정적 호스팅)
  │
  ▼
ALB (api.clmakase.click) ─── WAF (L7 필터링)
  │
  ▼
EKS Cluster (ap-northeast-2)
  ├─ oliveyoung-api Pod × 1~100 (Istio sidecar 포함)
  │    └─ KEDA ScaledObject (Kafka lag / Datadog RPS / Cron)
  ├─ Kafka 3-Broker StatefulSet
  │    └─ Zookeeper (broker 등록·리더 선출·offset 관리)
  ├─ Karpenter (c/m/r 6세대+, Spot 자동 프로비저닝)
```

```
├─ ArgoCD (GitOps selfHeal)
└─ Istio + Kiali (mTLS, 서비스 메시 가시화)
   │
   ├─ Aurora MySQL (Multi-AZ, HikariCP pool 5)
   └─ ElastiCache Redis (대기열 상태 캐싱)
```

핵심 기술 스택 선택 근거 **(Trade-off 포함)**

기술	선택 근거	포기한 것
Kafka 3-Broker	`productId` 파티션 키 → 동일 상품 순서 보장 + 병렬 처리. Non-blocking Retry 다단계 격리를 코드 레벨에서 완전 제어. Replay 가능한 메시지 보존	SQS 대비 운영 복잡도 ↑. StatefulSet·broker ID 고정·replication factor 직접 관리
Karpenter	EKS Managed Node Group 3회 `NodeCreationFailure` 후 전환. 인스턴스 패밀리 자동 선택으로 전량 Spot 운용 가능	Fargate "닭과 달걀" 부트스트랩 문제 직접 해결. 16개 연쇄 에러 감수
KEDA 복합 트리거	Kafka lag + Datadog RPS + Cron 3종 조합. 비즈니스 지표 반응형 스케일링	Deployment `replicas` 필드와 충돌 — 필드 완전 제거 필수라는 비자명한 제약
ArgoCD GitOps	`selfHeal + prune`으로 드리프트 자동 복구. commit SHA 태그 연동으로 정확한 버전 추적	`selfHeal`이 Secret placeholder 덮어쓰는 부작용 → Secret 관리 경계 설계 필수
Terraform 16개 모듈	VPC → EKS → RDS → WAF → CloudFront 전체 재현 가능한 IaC	모듈 간 순환 참조(RDS ↔ Secrets) 설계 실수 시 `apply` 불가
Istio mTLS	Zero Trust 서비스 간 통신 암호화. Kiali로 메시 토폴로지 실시간 가시화	istio-proxy 사이드카 리소스 추가 소비 — 고트래픽 시 별도 리소스 계획 필요

🟢 핵심 트러블슈팅

TS-1. Kafka 아키텍처 결정 — 브로커 장애 시 P95 87% 개선

Problem Version A(단일 브로커 + Circuit Breaker) 환경에서 브로커 강제 종료(`docker stop kafka`) 시뮬레이션 결과, P95가 3,137ms로 급등하고 주문 데이터 유실 발생.

Analysis

브로커 *1*대 다운 장애 시뮬레이션 실측 *(100 Users):*

지표	**Version A**	**Version C**	차이
Throughput	0.4 req/s	3.3 req/s	C가 **8.3**배
Avg Latency	1,396ms	312ms	C가 **4.5**배 빠름
P95 Latency	**3,137ms**	**436ms**	**C**가 **7.2**배 빠름
P99 Latency	3,982ms	485ms	C가 8.2배 빠름
주문 데이터	유실	무손실	—

정상 트래픽 실측 비교*:*

Users	**Version A Avg**	**Version C Avg**	개선율
100명	1,669ms	1,530ms	8%
1,000명	**439ms**	**303ms**	**31%** 단축
3,000명	325ms	355ms	유사

Circuit Breaker는 서비스 자체는 살리지만 Redis fallback 경로로 전환되면서 지연 8배 상승. 단일 브로커(SPOF)가 다운되면 Kafka 경유 주문 이벤트는 유실. "서비스는 살지만 매출 데이터가 없는" 최악의 시나리오.

Action

3-Broker 클러스터 구성:

- `replication.factor=3`, `min.insync.replicas=2`
- Zookeeper가 heartbeat 단절 감지 → 팔로워 중 리더 자동 선출 → 브로커 1대 다운에도 서비스 유지
- `productId`를 파티션 키 → 동일 상품 내 순서 보장 + 다른 상품 병렬 처리

Non-blocking Retry 3단계 설계 (`@RetryableTopic`):

Stage	토픽	지연	상정 원인
0	`order-events-retry-0`	1초	네트워크 순간 오류
1	`order-events-retry-1`	5초	DB 커넥션 부족
2	`order-events-retry-2`	30초	심각한 인프라 장애
DLT	`order-events.DLT`	—	수동 Replay 대기

실패 메시지가 별도 retry 토픽으로 격리 → 원본 파티션의 정상 메시지 처리 미중단. Blocking 방식이었다면 1건 실패 시 파티션 전체 정지(Head-of-Line Blocking).

Micrometer Counter 7종 삽입:

order_success_total — 정상 처리

order_retry_total{stage="0"} — 네트워크 지터 감지

order_retry_total{stage="1"} — DB 병목 감지

order_retry_total{stage="2"} — 인프라 장애 감지

order_dlt_total — 긴급 대응 트리거

stage별 카운터 상승 패턴만으로 장애 계층 즉시 식별 가능하도록 설계.

Spring Profile `version-a` / `version-c`로 전환 가능한 비교 실험 구조. 동일 인프라에서 아키텍처 차이만 변수로 격리.

Result

- 브로커 장애 P95: **3,137ms → 436ms (87%** 개선**)**
- 정상 1,000 VU P95: **859ms → 431ms (50%** 개선**)**
- 주문 데이터 완전 무손실 (DLT Replay 가능)
- Git: `6e592dc` (feat: Spring profile version-c 활성화)

TS-2. KEDA 스케일링 설계 실패 — replicas 충돌과 Cold Start 문제

Problem KEDA `ScaledObject`를 배포하고 부하테스트를 시작했지만 트래픽이 급증해도 **Pod** 수가 의도한 대로 늘지 않았다**.** KEDA ACTIVE: True 상태인데 실제 Pod은 고정된 상태.

추가로 세일 오픈 순간(T+0) 직후 초기 몇 분간 응답 지연이 두드러지는 문제도 발생.

Analysis

문제 **1 — KEDA ↔ Deployment replicas** 충돌

`deployment.yaml`에 `replicas: 2`가 명시된 상태에서 KEDA가 ScaleUp 명령을 내리면, Kubernetes Deployment Controller가 이를 인식하지 못하고 `replicas: 2`로 지속 복원. KEDA와 Deployment Controller가 같은 필드를 두고 서로 덮어쓰는 상태.

KEDA → replicas: 25 로 변경 시도

↓

Deployment Controller → "desired state는 2" → replicas: 2 로 복원

↓

KEDA → replicas: 25 로 변경 시도

↓

... (무한 반복)

`Deployment.spec.replicas`는 KEDA(HPA) 사용 시 반드시 제거해야 한다. HPA가 단독으로 이 필드를 소유해야 충돌이 사라진다.

문제 **2 — Cold Start**와 **Reactive Scaling**의 한계

스케일링은 항상 반응형(Reactive)이다. `minReplicas: 2`이면:

- 세일 오픈 → 트래픽 급증 → KEDA 감지 → ScaleUp 명령 → 노드 프로비저닝(Karpenter 2~3분) → Pod 기동 → Ready
- 이 과정이 완료되기 전, 세일 오픈 첫 수십 초가 가장 **Pod**이 부족한 순간
- 세일 오픈 직후 초기 사용자 경험이 최악이 되는 역설

Action

문제 **1** 해결**:**

```
# deployment.yaml — replicas 필드 완전 제거
spec:
  # replicas: 2  ← 이 줄 삭제
  selector:
    matchLabels:
      app: oliveyoung-api
```

KEDA ScaledObject가 HPA를 생성하고 단독으로 replicas를 제어하도록 위임.

문제 **2** 해결 **— Warm-up** 전략**:**

```
# keda/scaled-object.yaml
spec:
  minReplicaCount: 10   # 2 → 10 (세일 오픈 전 사전 준비)
  maxReplicaCount: 100
  triggers:
    - type: kafka        # Kafka consumer lag
    - type: datadog      # Datadog RPS
    - type: cron         # 세일 오픈 시간대 사전 스케일업
```

Cron 트리거로 세일 오픈 시각 전에 minReplicas를 10으로 올려 Pod Warm-up 완료 상태 유지. 첫 트래픽 유입 시 이미 10개 Pod이 Ready 상태.

스케일업 속도도 `10개/30s → 50개/30s`로 상향하여 급격한 트래픽 상승에 신속 대응.

Result

T+0:57 k6 VU 상승 시작 — 이미 10개 Pod Ready (Cold Start 없음)

T+5:45 KEDA 발동 — 10 → 86 Pod (50개/30s 정책, 약 30초 이내)

T+6:xx 100 Pod 도달 (maxReplicas 한도)

T+8~20 150,000 VU 피크 13분 안정 유지

- Pod 스케일: 10 → 100개 약 **60**초 이내
- 세일 오픈 직후 Cold Start **0**건
- 스케일링 충돌 해소 → KEDA 단독 제어 확립
- Git: `0b9b0ab` (fix: Deployment replicas 제거 — KEDA HPA 제어 충돌 방지)

  참고: 병행하여 발생한 istio-proxy OOMKilled는 limit 256Mi → 10Gi 상향과 request/limit 분리(request: 128Mi)로 해결. 150,000 VU 피크 시 OOMKilled 0건 달성. 이는 아키텍처 결정이 아닌 운영 리소스 튜닝 항목으로 인프라 역량 섹션에 기록.

TS-3. EKS 노드 프로비저닝 3회 실패 → Karpenter 전환 + 16개 연쇄 에러

Problem `aws_eks_node_group` Terraform Apply 후 `NodeCreationFailure` 3회 연속. 노드가 클러스터에 조인되지 않아 모든 워크로드 Pod이 `Pending` 상태.

Analysis 4가지 방법 순차 시도:

1. Launch Template에 클러스터 SG + 노드 SG 적용 → 실패
2. `block_device_mappings` 이동 → 실패

3. `ami_type` AL2023 → AL2, `cluster_version` 1.29 → 1.30 → 실패
4. 원인 추적: 팀이 이미 Karpenter로 노드 관리를 전환한 상태에서 Managed Node Group을 병행 시도 → 구조적 충돌

Action

`aws_launch_template.eks_nodes`, `aws_eks_node_group` 완전 제거. Karpenter 설치 과정에서 **16**개 연쇄 에러 순차 해결**:**

#	에러	원인	해결
1	`failed calling webhook "mservice.elbv2.k8s.aws"`	노드 없어 ALB Controller webhook endpoint 없음	mutating/validating webhook 수동 삭제
2	CRD `EC2NodeClass` not found	구버전 Helm repo(v0.16.x)가 v1alpha5만 지원	OCI registry에서 v1.0.1 재설치
3	CoreDNS `Pending` → DNS 전체 불가	노드 없음 → CoreDNS 스케줄링 불가	CoreDNS용 Fargate Profile 생성 (닭과 달걀 해결)
4	Karpenter Pod `Pending`	노드 없음 → Karpenter 자신도 스케줄링 불가	karpenter NS용 Fargate Profile 생성
5	`eks:DescribeCluster` AccessDeniedException	Karpenter Controller Role EKS 권한 없음	IAM inline policy 추가
6	`AWS.SimpleQueueService.NonExistentQueue`	SQS Queue 미사용인데 `interruptionQueue` 설정	Helm values에서 제거
7	`consolidateAfter cannot be combined with WhenUnderutilized`	NodePool spec 문법 오류	`consolidateAfter` 제거
8	Karpenter v0.37 vs v1.0.4 CRD 충돌	2개 버전 동시 설치로 CRD 버전 불일치	v1.0.1 단일 버전으로 통일

...	+8개 추가 에러	SG 중복, CIDR 충돌, ENI 블로킹 등	단계별 추적·해결

Fargate Profile로 Karpenter와 CoreDNS를 부트스트랩. 이후 본 워크로드는 Karpenter가 프로비저닝한 EC2 Spot 노드로 이동.

Result

- 노드: **2**대 → 최대 **26**대 자동 프로비저닝 (신규 23개)
- Pod: **1** → **100**개 자동 스케일아웃
- 인스턴스 전량 Spot (c8i-flex, m7i-flex, r8i 혼합) → 비용 최적화
- Karpenter v1.0.1 안정화, EC2NodeClass v1 API 통일
- Git: `7c9a3fe`, `fb9fe12`, `8899072`

TS-4. Aurora "Too many connections" — DB 연결 수 계산 공식 도출

Problem 부하테스트 중 API 500 응답 급증. `Aurora Too many connections` 에러로 DB 연결 전면 거부. KEDA가 Pod을 늘릴수록 상황이 악화.

Analysis

총 DB 연결 수 = 현재 Pod 수 × HikariCP pool_size

= 20 Pod × 10(기본값) = 200 연결

Aurora t3.medium 기본 `max_connections` ≈ 90. KEDA가 Pod을 늘릴수록 DB 연결이 선형 증가 → Aurora 한계 초과 → 연결 거부 → 더 많은 에러 → KEDA가 더 많은 Pod 투입 → 악순환.

Pod 증설이 장애를 해소하는 게 아니라 오히려 심화시키는 구조. 스케일 계획과 DB 연결 버짓을 연결한 공식이 없었던 것이 근본 원인.

Action

공식 도출 후 역산:

maxReplicas × pool_size ≤ Aurora max_connections

→ max_connections(90) / maxReplicas(100) = 0.9 → pool_size 최대 1

(현실적으로는 여유 포함하여 pool_size: 5로 설정)

→ 검증: 100 Pod × 5 = 500 > 90 (여전히 초과)

→ 해결: minReplicas 조정 + pool_size: 5 + Aurora 인스턴스 업그레이드 검토

```
# deployment.yaml
env:
  - name: SPRING_DATASOURCE_HIKARI_MAXIMUM_POOL_SIZE
    value: "5"   # 기본값 10 → 5 축소
```

Result

- DB 연결 한계 초과 장애 재발 **0**건
- 스케일 계획 수립 시 DB 연결 버짓 사전 계산 프로세스 팀 내 공유
- "maxReplicas 늘리기 전에 pool_size 먼저 확인" 체크리스트 수립
- Git: `c33b434`

TS-5. GitOps 두 가지 실패 — Secret 경계 설계와 CI/CD 연결 고리

5-A. ArgoCD selfHeal이 실제 Secret을 덮어쓰는 문제

Problem KEDA `ScaledObject READY: False`. `kubectl get secret datadog-keda-secret` → `DATA=0` 반복. Secret을 주입해도 곧 사라짐.

Analysis `trigger-auth-datadog.yaml`에 Secret 리소스 블록을 함께 정의 → ArgoCD `selfHeal`이 git 상태(placeholder `<DATADOG_API_KEY>`)로 지속 복원 → 실제 주입된 키가 계속 덮어씌워짐.

Action

```
# trigger-auth-datadog.yaml에서 Secret 블록 완전 제거
# (git에 Secret을 절대 정의하지 않는다)

# .gitlab-ci.yml — 전용 job에서만 주입
deploy-secrets:
  script:
    - kubectl create secret generic datadog-keda-secret \
        --from-literal=apiKey=$DATADOG_API_KEY \
        --dry-run=client -o yaml | kubectl apply -f -
    - kubectl rollout restart deployment/keda-operator -n keda
```

원칙 수립: **ArgoCD**가 모르는 리소스 **= ArgoCD**가 건드리지 않는다**.**

5-B. "파이프라인 성공 ≠ 배포 완료" — CI/CD 연결 고리 누락

Problem CI 파이프라인 전 단계 Success. 그러나 ArgoCD UI에서 최신 커밋 Author는 갱신되지만 실제 Pod 이미지는 교체되지 않음. Datadog에도 변경사항 미반영.

Analysis `latest` 태그 사용 → `deployment.yaml` YAML 텍스트가 불변 → ArgoCD "감지할 변경 없음" 판단. CI(빌드)와 CD(배포) 사이의 Manifest Update 연결 고리 누락. ArgoCD UI의 Author 갱신은 Git 메타데이터 반영일 뿐, 런타임 변경을 보장하지 않음.

Action

```
update-manifest:
  stage: deploy
  script:
    - NEW_TAG=$CI_COMMIT_SHORT_SHA
    - sed -i "s|image: .*|image: ${ECR_URL}:${NEW_TAG}|g" k8s/deployment.yaml
    - git add k8s/deployment.yaml
    - git commit -m "ci: update image tag to ${NEW_TAG} [skip ci]"
    - git push origin main
    # [skip ci] → 무한 파이프라인 루프 방지
```

GitOps 원칙 확립:

- 소스코드 빌드 성공 ≠ 실서비스 배포 완료
- 모든 인프라 상태는 Git Desired State와 동기화되어야 함
- commit SHA 태그 = 정확한 버전 추적 + 롤백 지점

Result (5-A + 5-B 합산)

- KEDA `ScaledObject READY: True / ACTIVE: True` 정상화
- 코드 Push → 자동 배포까지 파이프라인 완전 자동화
- "Secret은 git에 정의하지 않는다" + "이미지 태그는 SHA" 두 가지 팀 정책 수립
- Git: `07bb36c`, `bf2d7ee`, `2b069eb`

🟡 클라우드 및 인프라 역량

Terraform IaC — 16개 모듈 전체 소유

```
terraform/
├── main.tf              # 전체 모듈 오케스트레이션
├── karpenter_iam.tf     # Karpenter IAM (OIDC Trust, Node KMS)
└── modules/
    ├── vpc/             # 6서브넷, NAT GW, Karpenter discovery 태그
```

```
├── security-groups/    # 7개 SG (ALB/CP/Node/RDS/Redis/CLI/VPC EP)
├── eks/                # 클러스터 + OIDC (Node Group 제거)
├── ecr/                # oliveyoung-api 레포지토리
├── rds/                # Aurora MySQL Multi-AZ
├── elasticache/        # Redis
├── alb-controller/     # Helm 릴리스
├── argocd/             # Helm 릴리스
├── cli/                # SSM 기반 Bastion
├── secrets/            # Secrets Manager (순환참조 해결)
├── waf/                # WAF (L7 필터링)
├── kms/                # KMS 데이터 암호화
├── s3/                 # 프론트엔드 정적 호스팅
├── route53/            # DNS
├── acm/                # SSL 인증서
└── cloudfront/         # CDN
```

Multi-AZ 설계 **— VPC** 서브넷 **2**개 → **6**개 확장**:**

서브넷	**CIDR**	용도
Public × 2	10.0.101~102.0/24	ALB, NAT GW
Private App × 2	10.0.1~2.0/24	EKS 워커 노드
Private Data × 2	10.0.11~12.0/24	Aurora, ElastiCache

모듈 의존성 핵심 해결**:**

- RDS ↔ Secrets 순환 참조 → `db_host`를 Secrets에서 제거, deployment.yaml 직접 참조
- 중복 `aws_secretsmanager_secret` 리소스 → root 레벨 제거
- `outputs.tf` vs `output.tf` 중복 파일 → 기존 파일 유지

CI/CD — 8단계 완전 자동화 파이프라인

```
git push main
  ↓ [1] test          → Gradle JUnit (allow_failure)
  ↓ [2] build         → Docker → ECR (commit SHA 태그)
  ↓ [3] trivy-scan    → CVE 취약점 스캔 (Tomcat 10.1.45, CVE-2025-55754 패치)
  ↓ [4] update-manifest → sed로 SHA 교체 → git push [skip ci]
```

↓ [5] deploy-secrets → KEDA Secret CI 단독 주입 (git 제외)

↓ [6] deploy-frontend → npm build → S3 → CloudFront 캐시 자동 무효화

↓ [7] load-test → k6 (when: manual, allow_failure: true)

↓

ArgoCD 감지 → EKS 롤링 배포

설계 포인트:

- `[skip ci]` 태그로 manifest update 커밋이 파이프라인을 재트리거하는 무한루프 방지
- `allow_failure: true` + `when: manual`로 부하테스트 실패가 전체 파이프라인을 차단하지 않음
- CloudFront 캐시 무효화 자동화로 Mixed Content 재발 방지

150,000 VU 최종 부하테스트 결과 (2026-02-26)

테스트 시나리오: 0 → 150k VU 2분 급상승 → 13분 피크 유지 → 5분 감소 (총 20분) **k6** 구성: parallelism 10 (파드당 15,000 VU, 8core/16GB)

지표	결과
OOMKilled	**0**건
CrashLoopBackOff	**0**건
비정상 RESTART	**0**건
API 파드 최대	**100**개 (KEDA maxReplicas)
Karpenter 노드 최대	**26**개 (신규 23개)
서비스 중단	없음
총 vCPU	~128 core
총 RAM	~600GB
인스턴스 타입	전량 **Spot**

Peak RPS	56,300 hits/s (Datadog 실측)
Stable RPS	49,500 hits/s (피크 구간 평균)
총 처리 요청	4.65M hits (20분)
Success Rate	100% (5xx 에러 0건)
P99 Latency	180ms 이하

1차→**3**차 개선 이력 **(OOMKilled 0**건까지**):**

항목	**1**차	**2**차	**3**차 최종
OOMKilled	다수	다수	**0**건
초기 Pod	2개	2개	**10**개 **Warm-up**
KEDA scaleUp 속도	10개/30s	10개/30s	**50**개**/30s**
istio-proxy limit	256Mi	2Gi	**10Gi**
Deployment replicas	2 고정	2 고정	제거

보안 설계

- **WAF**: ALB 앞단 L7 필터링
- **KMS**: 데이터 암호화 키 관리
- **Istio mTLS**: Zero Trust 서비스 간 통신 암호화
- **Trivy**: CI 파이프라인에 CVE 스캔 자동화 (Tomcat 취약점 패치 이력)
- **Secrets Manager**: DB 비밀번호 자동 생성 + 순환
- **SSM Bastion**: 직접 SSH 없이 CLI Server 접근

🔴 성장 포인트 및 회고

증명된 엔지니어링 역량

역량	근거
데이터 기반 아키텍처 결정	A/C 실측 비교표, `maxReplicas × pool_size` 공식으로 모든 설계 정당화
Availability vs Consistency 트레이드오프 인식	Non-blocking Retry 도입 시 메시지 순서 역전 가능성을 사전 인지하고 `productId` 파티션 키로 동일 상품 순서 보장
장애 연쇄 추적	Karpenter 전환 시 16개 연쇄 에러를 단계별로 원인 파악·해결. 에러→원인→해결→예방 4단계 문서화
GitOps 경계 설계	ArgoCD 관리 대상(Deployment/Service)과 비관리 대상(Secret)의 경계를 실패를 통해 정립하고 팀 정책으로 수립
3차 반복 개선 프로세스	OOMKilled 반복 → 가설 수립 → 테스트 → 재분석의 체계적 사이클로 최종 0건 달성
IaC 전체 소유	VPC부터 CloudFront까지 16개 모듈 직접 작성·디버깅. 인프라 변경이 git 이력으로 100% 추적 가능

Technical Debt — 다음 단계 개선 항목

1. **Datadog APM** 미완성: KEDA 트리거는 동작하나 Micrometer 7종 메트릭 실수집 불가. Datadog Agent DaemonSet 배포 → `/actuator/prometheus` openmetrics 연동 완성 필요.

2. **k6 Summary** 유실: 최종 150k VU 테스트를 `kubectl apply` 직접 실행 → Pod 삭제 후 `k6-summary.json` 유실. 다음에는 GitLab CI Artifact 보존 설정 필요.

3. **Karpenter Spot Interruption Handler** 미설치: `interruptionQueue` 제거 상태. Spot 회수 2분 전 SIGTERM 신호를 SQS로 받아 graceful drain하는 흐름이 없음. SQS 기반 Interruption Handler 재도입 필요.

4. **IaC** 드리프트 잔존: Fargate Profile, Karpenter IAM inline policy 수동 생성 리소스가 Terraform State 미편입. `terraform import` 또는 신규 모듈화 필요.

5. **Zookeeper → KRaft** 마이그레이션: Kafka 3.x KRaft 지원으로 Zookeeper(현재 SPOF) 제거 가능. 다음 버전 검토.

🎤 면접 **Q&A —** 실측 수치 기반

질문	답변
"Version A/C 차이가 뭔가요**?"**	브로커 장애 시 A: P95 3,137ms·주문 유실 / C: P95 436ms·무손실. "서비스는 살지만 매출 데이터 없음" vs "서비스도 살고 데이터도 안전"
"왜 **SQS** 안 쓰고 **Kafka** 직접 구축했나요**?"**	① 파티션 병렬 처리로 고트래픽 수평 확장 ② Non-blocking Retry 다단계 격리를 코드 레벨에서 완전 제어 ③ Replay 가능한 메시지 보존. 운영 복잡도 트레이드오프는 인지하고 선택
"최대 **RPS**가 얼마였나요**?"**	150k VU × 평균 7.5 req/iteration ÷ 10s = 약 **10**만**~11**만 **RPS** 추정. *"Datadog `as_rate()` 실측 기준 피크 56,300 hits/s, 안정 구간 평균 49,500 hits/s, 총 4.65M 요청 처리. 이론 최대치(~112k)보다 낮은 이유는 고부하 시 응답시간 증가로 k6 iteration이 10s → 20s 이상으로 연장된 것과 Istio-proxy 사이드카 오버헤드가 복합 작용한 결과입니다."*
"파드 **100**개가 얼마나 빨리 떴나요**?"**	KEDA 발동 후 약 **60**초 이내 **100**개 도달 (50개/30s 정책). 노드 프로비저닝 포함 약 3분
"OOM은 왜 났나요**?"**	istio-proxy가 모든 인바운드/아웃바운드 트래픽을 가로채는 사이드카 구조상 10만 RPS 수준에서 256Mi 한도 초과. limit/request를 분리하여 limit 10Gi로 피크 흡수
"Aurora 연결 문제는요**?"**	`maxReplicas × pool_size ≤ max_connections` 공식 도출 후 pool 10→5 축소. 스케일아웃이 장애를 악화시키는 구조를 공식으로 사전 차단
"KEDA가 안 동작한 이유가요**?"**	`Deployment.spec.replicas` 고정값이 HPA 명령을 덮어씀. 이 필드는 KEDA 사용 시 반드시 제거해야 HPA가 단독 제어권을 가짐
"Datadog 메트릭은요**?"**	Micrometer Counter 7종 코드는 완성. Datadog Agent 미배포로 실수집 미완. 파이프라인 설계는 `/actuator/prometheus` → openmetrics check 구조로 완성되어 있음
"노드는 자동으로 줄어드나요**?"**	Karpenter `WhenUnderutilized` 정책으로 유휴 상태 지속 시 자동 회수 및 통합(Consolidation)

최종 한 문장

"150,000 VU OliveYoung 세일 러시를 Kafka 3-Broker Non-blocking Retry로 주문 데이터 무손실 보장, Karpenter Spot으로 Pod 1→100·노드 2→26 자동 스케일링으로 Datadog 실측 56,300 RPS·P99 180ms·Success Rate 100% 를 달성하며 데이터 기반 인프라 의사결정 능력을 증명했습니다."

5만 RPS가 낮아 보이는 문제

숫자 자체가 약한 게 아니라 맥락 없이 던지면 약해 보이는 겁니다. 순서를 바꾸면 됩니다.

```
❌ "RPS가 초당 5만이었습니다"
✅ "150,000명이 동시 접속한 상황에서
    5xx 에러 0건, P99 180ms 이하,
    4.65M 요청을 전량 처리했습니다.
    이때 측정된 RPS가 약 56,000 hits/s입니다."
```

결과(안정성)를 먼저 말하고 RPS는 그 증거로 제시하는 구조입니다.

이론치와 실측치 gap 설명은 그대로 써도 됩니다

오히려 이 설명이 면접에서 가장 임팩트 있는 부분이 될 수 있습니다.

"기댓값 11만과 실측 5만의 차이를 분석해서 iteration 지연과 Istio 오버헤드로 원인을 파악했습니다."

이게 단순히 숫자를 읊는 게 아니라 시스템 동작 원리를 이해하고 있다는 증거가 됩니다.

"더 최적화 안 했냐?"에 대한 답변은 한 가지만 수정

지금 말씀하신 "다른 활동이 중요하다고 판단" 프레이밍은 살짝 위험합니다. 면접관 입장에서는 "성능 문제를 회피했다"로 들릴 수 있습니다.

```
❌ "다른 활동이 더 중요하다고 판단했습니다"

✅ "목표인 150,000 VU 상황에서 에러 0건과 P99 180ms를
    달성했기 때문에 시스템 설계 목표는 충분히 검증됐다고
    판단했습니다. RPS 극대화보다 가용성과 무손실이
    이 프로젝트의 핵심 지표였습니다."
```

목표를 달성했기 때문에 추가 최적화를 하지 않은 것이지, 시간이 없어서가 아니라는 프레이밍입니다.